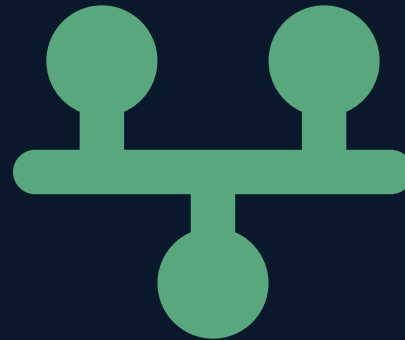


PORTADA

Arquitectura de Software

Semana 2 — Evolución de la Arquitectura de Software (Tecnologías y Frameworks)

Universidad de Antioquia · Ingeniería de Sistemas · 2554608 · 2026-2



APERTURA

**De una sola aplicación
a un framework para todos**



CASO · FICHA TÉCNICA

Ruby on Rails, 2004

Autor	David Heinemeier Hansson (DHH), en 37signals (hoy Basecamp)
Origen	Extraído del código real de la aplicación Basecamp , no diseñado desde cero en abstracto
Lanzamiento público	Julio de 2004
Principios centrales	Convención sobre configuración y DRY (Don't Repeat Yourself)
Impacto	Popularizó el patrón MVC en la web e influyó directamente en frameworks posteriores como Django (Python) y Laravel (PHP)

CASO · EL PROBLEMA

Antes de los frameworks: repetir todo, siempre

A finales de los 90, construir un sitio web dinámico significaba escribir manualmente, una y otra vez para cada proyecto: la conexión a la base de datos, el mapeo de filas a objetos, las rutas de cada página, la validación de formularios.

- **CGI (1993)**: scripts sueltos, uno por cada acción, sin estructura común.
- **Servlets y JSP (1997, Java)**: más ordenado, pero con mucho código repetitivo (boilerplate) por cada nueva funcionalidad.
- Cada equipo reinventaba su propia forma de organizar el proyecto — sin convenciones compartidas entre empresas.

CASO · LO QUE RAILS CAMBIÓ

"Convención sobre configuración"

En vez de que cada desarrollador decidiera —y configurara a mano— cómo se llama cada archivo, dónde va cada cosa y cómo se conecta la base de datos, Rails impuso **convenciones por defecto**: si el desarrollador las sigue, casi no hay que configurar nada.

- Un modelo llamado `Usuario` se mapea automáticamente a una tabla `usuarios`, sin escribirlo explícitamente.
- El *scaffolding* genera automáticamente el código base de una funcionalidad CRUD completa.
- Una demo pública de DHH en 2004 mostró crear un blog funcional en **15 minutos** — el video se volvió una referencia citada en la comunidad de desarrollo web.

CASO · EL IMPACTO

Una idea que se replicó en otros lenguajes

Rails no solo tuvo éxito propio: **cambió las expectativas de toda la industria** sobre cómo debía sentirse un framework web productivo.

- **Django** (Python, 2005) adoptó ideas muy similares de convención y ORM automático.
- **Laravel** (PHP, 2011) se inspiró explícitamente en la elegancia de Rails.
- El propio ecosistema Java, más adelante, adoptó ideas de "convención sobre configuración" en frameworks como Spring Boot.

Rails demuestra que un cambio de arquitectura de framework —no una nueva funcionalidad de negocio— puede transformar la productividad de toda una industria.

TRANSICIÓN

Rails es un punto en una línea más larga

Rails resolvió un problema de su momento —demasiado código repetitivo en el backend web—, igual que otros frameworks, antes y después, resolvieron el problema de **su** momento.

Para entender por qué existen tantos frameworks hoy, hay que ver la evolución completa: qué fuerza empujó cada salto.

CONCEPTO

¿Qué es un framework?

Un **framework** es un conjunto de código reutilizable que define la **estructura general** de una aplicación y le dice al desarrollador dónde poner su propio código — a diferencia de una simple colección de funciones sueltas.

La diferencia clave con una librería es quién tiene el control del flujo del programa — se conoce como *inversión de control*: el framework llama al código del desarrollador, no al revés.

CONTRAEJEMPLO

Mito: "framework" y "librería" son lo mismo

	Librería	Framework
Quién llama a quién	Tu código llama a la librería	El framework llama a tu código
Control del flujo	Lo tiene el desarrollador	Lo tiene el framework
Ejemplo	Una función que formatea fechas	Rails, Spring, Angular

Usar una librería es una decisión de detalle. Adoptar un framework casi siempre es una decisión arquitectónica — condiciona toda la estructura del proyecto (concepto visto la semana pasada).

CONCEPTO

Línea de tiempo de tecnologías backend

Año	Tecnología	Qué resolvió
1993	CGI	Primera forma estándar de generar contenido dinámico en la web
1997	Servlets / JSP (Java)	Programas que viven en el servidor sin reiniciar por cada solicitud
2000	Struts (Java)	Primer MVC ampliamente adoptado para separar lógica y presentación
2003	Spring (Java)	Inyección de dependencias, menos acoplamiento entre componentes
2004	Ruby on Rails	Convención sobre configuración, productividad radical
2009	Node.js	JavaScript también en el backend, modelo asíncrono

CONCEPTO

Línea de tiempo de tecnologías frontend

Año	Tecnología	Qué resolvió
1991–1995	HTML + CSS	Documentos estáticos con formato, sin interactividad real
1995	JavaScript	Interactividad en el navegador sin recargar la página
2006	jQuery	Manipular la página sin lidiar con diferencias entre navegadores
2010	AngularJS	Aplicaciones de una sola página (SPA): el navegador ya no recarga entre pantallas
2013	React	Interfaz como función del estado; reutilización de piezas (componentes)



CONCEPTO

El patrón detrás de cada salto

Cada nueva tecnología no apareció por moda: resolvió una **fuerza concreta** que la anterior no podía sostener.

- **Escalabilidad:** más usuarios de los que la arquitectura anterior podía atender (el caso de Twitter, visto la semana pasada).
- **Productividad del desarrollador:** mismo resultado, con menos código repetitivo (el caso de Rails, visto hoy).
- **Cambios de hardware o red:** computadoras más rápidas, conexiones más veloces, dispositivos móviles.
- **Nuevos requisitos de experiencia de usuario:** interfaces que responden sin recargar la página completa.

TENSIÓN CONCEPTUAL

¿Siempre hay que usar lo más nuevo?

Rails, en 2026, ya no es "lo más nuevo" — pero Basecamp, la aplicación que lo originó, sigue corriendo sobre Rails hoy.

- Adoptar la tecnología más reciente tiene un costo: menos documentación madura, menos desarrolladores con experiencia, más riesgo de que cambie o desaparezca.
- Quedarse con una tecnología antigua también tiene costo: menos soporte, dificultad para contratar, deuda técnica acumulada.

¿Qué debería pesar más al elegir la tecnología de un proyecto nuevo: qué tan nueva es, o qué tan bien resuelve el problema concreto?



INTERACCIÓN

¿Qué fuerza resolvió?

Para cada tecnología, identifiquen qué fuerza de la diapositiva "El patrón detrás de cada salto" resolvió principalmente:

1. React (componentes reutilizables, interfaz como función del estado)
2. Node.js (JavaScript en el backend, modelo asíncrono)
3. Spring (inyección de dependencias)
4. AngularJS (aplicaciones de una sola página)

5 minutos · respondan en voz alta o en el chat

CONCEPTO · LO QUE VIENE

De frameworks sueltos a stacks completos

Con el tiempo, ciertas combinaciones de tecnologías —un lenguaje de backend, una base de datos, un framework frontend— se volvieron tan comunes que recibieron nombre propio: **LAMP**, **MEAN**, **MERN**, entre otros.

Estos stacks completos, con el detalle de cada capa, se estudian a fondo en la Semana 5 de esta unidad ("Stack tecnológico").

SÍNTESIS

Ideas clave de hoy

1. Un **framework** invierte el control del flujo del programa; una librería no — esa diferencia lo vuelve una decisión arquitectónica.
2. **Ruby on Rails (2004)** demostró que un cambio de framework puede transformar la productividad de toda una industria.
3. Backend y frontend evolucionaron en paralelo, cada uno resolviendo las limitaciones técnicas de su momento.
4. Cada salto tecnológico responde a una fuerza concreta: **escalabilidad, productividad, hardware/red o experiencia de usuario**.
5. "Más nuevo" no es sinónimo de "mejor para este proyecto" — la elección tecnológica exige criterio, no moda.

INSTRUCCIONES

Taller de hoy — Implementación de ejemplo

Demostración guiada por el docente — 30 a 40 minutos

El docente muestra en vivo, sobre el mismo backend Spring Boot usado en sesiones anteriores, **cómo la inyección de dependencias** —el aporte central de Spring a la línea de tiempo de hoy— evita repetir la creación manual de objetos.

INSTRUCCIONES · EJEMPLO COMENTADO

Inyección de dependencias, en vivo

```
// Sin inyección de dependencias: el controlador
// crea y administra sus propias dependencias
public class SaludoController {
    private SaludoService servicio = new SaludoService(new SaludoRepository());
    // el controlador queda acoplado a implementaciones concretas
}

// Con inyección de dependencias (Spring):
// el framework crea y entrega las dependencias
@RestController
public class SaludoController {
    @Autowired
    private SaludoService servicio; // Spring la inyecta automáticamente
}
```

El framework, no el desarrollador, decide cuándo y cómo se crean los objetos. Esta es la "inversión de control" mencionada en la diapositiva del mito framework vs. librería.

No hay entregable de estudiantes: el objetivo es reconocer la inversión de control cuando aparezca en cualquier framework.

CIERRE

Para la próxima semana

- Identificar qué framework(s) usa el proyecto de Fábrica Escuela del propio equipo y a qué "salto" de la línea de tiempo corresponde.
- Repasar la diferencia entre librería y framework — se retoma en la Semana 3.

La Semana 3 aborda Diseño vs. Arquitectura y la integración de UML para generar vistas arquitectónicas — el primer puente entre la teoría de estas dos semanas y la práctica de modelar un sistema.

